ELI — Inference And Hardware

ELI v2.5.16

October 2026

Contents

ELI Inference & Hardware Boot **1**

Inference (cognition/gguf_inference.py, 3.2k LOC + inference_broker.py) . . . 1
Hardware profiling (core/hardware_profile.py, ~1400 LOC) . . . 2
GPU pack (packaging/pyinstaller/eli_gpu_pack.py, core/gpu_pack_runtime.py) . . . 4
Load-probe cache (core/load_probe.py) . . . 4
Settings (core/runtime_settings.py, ~1150 LOC) . . . 5
Paths (core/paths.py, 808 LOC) . . . 5
STT VRAM awareness . . . 6
Piper and LoRA device selection . . . 6
Honest assessment . . . 6

ELI Inference & Hardware Boot

> **Updated for v2.4.84.** GGUF path is canonical (optional Ollama backend still exists as a secondary path). VRAM fit reads {arch}.block_count from the GGUF header (model-agnostic) with a size heuristic fallback only when metadata is unreadable. Token budgets scale by reasoning mode via reasoning_modes.py.

How ELI loads a model, talks to it, and adapts to whatever machine it's on. The inference path is model-agnostic (see memory eli-model-agnostic); the boot path is hardware-adaptive. Files in eli/cognition/ and eli/core/.

Inference (cognition/gguf_inference.py, 3.2k LOC + inference_broker.py)

- **Model resolution (get_model_path)**: ELI_GGUF_MODEL_PATH env → model_path/custom_model_path/bundled_model_p settings keys. No baked model; empty default.
- **load_model(force_reload)**: resolves n_ctx (env → settings → config.get_gguf_n_ctx()), caps it to vision_coresident_text_ctx when a co-resident vision model is loaded; resolves n_gpu_layers similarly.
- **Graceful GPU-layer fallback**: if context allocation fails at the requested layer count, it retries with fewer layers / without flash-attention rather than crashing — keeps the model + n_ctx and degrades GPU offload instead.
- **Chat templating is family-aware**: _is_mistral_model / _is_chatml_model / _is_llama_model sniff the filename to pick the right prompt format. This is *adaptation* to whatever model you load (like the ctx table), not a hardcoded model — but it is filename-based, so an unrecognised naming scheme falls back to a default template.

- **Serialization**: all calls hold `_LLM_CALL_LOCK` (a native RLock from `runtime/native_locks`) — llama_cpp is not safe under concurrent calls; this is also what vision hot-swap and the ambient daemon coordinate on.
- **The model's state is kept at the end of a prompt's fixed part**: `_format_prompt` notes where the part that does not depend on the message ends (the system prompt, or the intent resolver's action list and examples). Before a call, `_resume_from_saved_state` gives back the state saved there by an earlier call that started the same way, or reads that part alone and saves the state (llama.cpp's per-sequence state calls: the model's memory only, no logits). The model then reads only what follows. A hybrid model (`qwen35moe`: most layers carry a running state, not an attention cache) cannot be rolled back to a shared prefix, so without this every call read its whole prompt: measured on Qwen3.6-35B-A3B, an RTX 2060 SUPER and experts in RAM, Normal mode's second pass took 63.8 s and took 3.2 s with the state given back (the same 60-token answer both ways); the intent resolver went from 26.6 s to 2.5 s after its first call. A 7k-token state is 135 MiB, saved in 0.08 s and restored in 0.02 s. The states are kept within an eighth of free memory and two contexts' worth of state, the oldest giving way; background work uses them but never adds one; they go when the model is unloaded. `ELI_PROMPT_STATE_REUSE=0` turns it off.
- **Prefill abort**: shutdown and cancel register `llama_set_abort_callback` so a long prompt prefill stops immediately — `StoppingCriteria` alone only runs between output tokens.
- **Live control**: `get_live_runtime_override`, `unload_model`, `reload_model` let the GUI swap models / change settings without a full restart.
- **`InferenceBroker`** (`inference_broker.py`): the higher-level `infer()` / `gguf_ready` abstraction the orchestrator, engine, and ReAct loop call, so callers don't touch `gguf_inference` directly.
- **CPU runtime preflight (`core/llama_cpu_compat.py`)**: before the first native load, a cached smoke test (once per llama.cpp build and CPU) checks the runtime starts on this CPU. Prebuilt wheels assume AVX-VNNI and SIGILL on older chips; the loader now reports that plainly instead of crashing. Only a definitive illegal-instruction blocks a load. `ELI_CPU_PREFLIGHT=0` disables it.

Hardware profiling (`core/hardware_profile.py`, ~1400 LOC)

Free-VRAM-aware sizing, genuinely cross-vendor: NVIDIA via `nvidia-smi`, then a kernel-driver fallback, then AMD via `rocm-smi`, then AMD via the stock `amdgpu` sysfs (the common desktop case where ROCm is absent), then discrete Intel Arc, then integrated Intel/AMD/Qualcomm/Apple unified memory. All of them populate the same `HardwareProfile` fields, so smart-fit GPU-layer allocation is identical whatever the card.

- **When the live VRAM probe can't run, fall back to a known-model lookup, not one flat number for every card.** `nvidia-smi` can fail for reasons that have nothing to do with whether a GPU is present or how big it is — most concretely, a loaded kernel module out of sync with the userspace driver library after an update with no reboot yet (confirmed in the field: NVML's own "Driver/library version mismatch", RTX 2060 SUPER, halved GPU layer counts). The model name is still readable straight from the kernel module's `/proc/driver/nvidia/gpus/*/information` even when NVML itself is broken, so `_nvidia_vram_mb_from_model_name()` looks up real factory VRAM for common cards (GTX 10xx through RTX 40xx) instead of guessing a flat 4096MB for a card that might have 24GB. Same idea for discrete Intel Arc, which has no free-VRAM sysfs at all (`_intel_arc_vram_mb_from_name()`, A310 through B580). An unrecognized model still falls back to a conservative flat guess — this narrows how often that happens, it doesn't eliminate the last resort.
- **Unified memory scales with actual RAM, not a flat 8GB ceiling.** `_estimate_integrated_vram_mb()` (Apple Silicon, AMD APUs, Intel iGPUs) used to cap its shared-VRAM budget at 8192MB regardless of how much RAM the machine had — a 64GB+ Mac Studio got the identical GPU-layer budget as an 8GB laptop iGPU. It now uses the same uncapped fraction-of-available-RAM basis as `cpu_ram_budget_mb()`, which it was already supposed to match.

- Subprocess calls to `nvidia-smi` / `rocm-smi` / `lspci` sanitize `LD_LIBRARY_PATH` first (`_external_tool_env()`) — a frozen PyInstaller build points that at its own bundle for its own libraries, and handing that to a *system* binary can make the real tool silently fail, landing on the same conservative-fallback path as a genuinely absent tool.
- `HardwareProfile` dataclass tracks **free** vs total VRAM (free is what matters for whether a profile actually loads).
- **`gpu_detection_uncertain`** — a real GPU PCI device can be present in `/sys/class/drm` (kernel sees it) while every vendor-specific probe above fails to characterize it (a timeout, a missing tool, a permission error, an unrecognised card). That used to collapse into the identical `has_gpu=False` as a genuinely GPU-less machine, so "no GPU" was reported with full confidence even when the honest answer was "couldn't check." `_linux_gpu_pci_device_present()` is the vendor-agnostic last-resort check that sets this flag; `has_gpu` itself stays `False` either way (still the safe default for offload decisions) — this field is for anything that *reports* the result.
- `_kv_cache_mb(n_ctx, n_layers)` — KV-cache cost, for the cache type the load will use.
- **KV cache type** — the settings `cache_type_k` / `cache_type_v` are empty (fp16, the default) or `q8_0`. Any other quantized type stored there reads, sizes and loads as `q8_0` (`core/gguf_sizes.py`: `cache_setting`, `effective_cache_type`): measured on llama.cpp 0.3.35, a `q8_0` cache answered the same as fp16 at the same speed on CUDA, Vulkan and the processor, and a `q4_0` cache answered wrongly. The tuner chooses `q8_0` only when an fp16 cache would not hold the context in the memory left after the weights. llama.cpp is given the type as `type_k` / `type_v` with `flash_attn` (`llama_cache_args`); arguments it does not take are dropped and logged (`model_load_diagnostics.llama_accepts`), a cache it refuses is loaded again as the default with the fit redone, and the type it reports is what the runtime snapshot records.
- `_compute_graph_reserve_mb(n_ctx, batch)` — the model-agnostic compute buffer estimate (`256MB + 24MB/1K ctx + 1.5MB/batch`), reserved so a profile that loads cleanly doesn't then hard-crash on the first decode when the lazy compute buffer pushes VRAM over the limit.
- `_layers_for_size`, `ModelRecommendation` — pick offload layers from model size and free VRAM.
- **Fit profiles** — startup exposes **Balanced / Max GPU / Max context** (`fit_priority` in settings, `ELI_FIT_PRIORITY` env): Balanced preserves context and sheds GPU layers → batch → ctx (default); Max GPU shrinks ctx/batch before dropping layers; Max context spills weights to RAM for larger windows. `unified_fit_config()` is the joint VRAM+RAM planner used by `recommend()`, the GUI load ladder, and `gguf_inference` smart-fit.
- **CPU/RAM fit when GPU offload is inactive**: `effective_use_gpu_layers(hw)` returns false when `llama_supports_gpu_offload()` is unavailable, when `compute_mode=cpu`, or when `ELI_FORCE_GPU_LAYERS=0`. When false, `recommend()` and the startup optimizer use `cpu_ram_fit_config()` — same `smart_fit_config` math, budgeted from live RAM instead.

Vendor parity leaks at the ~15 sites that bypass `hardware_profile` and re-implement `nvidia-smi` directly instead of reading the profile it already built — e.g. `perception/local_whisper_stt.py`'s `_gpu_total_mb()` and a few status-reporting surfaces (`self_status.py`, `truth_report.py`) that fall back to "GPU telemetry unavailable" on non-NVIDIA hardware they could otherwise read from the shared profile. The fix direction is routing those sites through `hardware_profile` rather than adding more vendor detection.

Install-time detection is a SEPARATE stack from `hardware_profile`

`hardware_profile` only runs once ELI itself is importable. Getting there — deciding whether to build `llama-cpp-python` with CUDA/ROCm/Vulkan in the first place — happens before that, in two more independent detectors that do not share its code at all: `install.sh` (bash, the actual build) and `eli/setup/hardware_policy.py` (Python, decides `ELI_INSTALL_CPU_ONLY` for `eli_setup.sh`'s GUI-wizard-or-terminal install). A bug fixed in one of the three does nothing for the other two — confirmed in the field, twice:

- **Both install-time detectors misread nvidia-smi's specific NVML "Driver/library version mismatch" failure as a working GPU.** That failure prints a first line matching the usual "broken driver" keyword filters ("Failed to initialize NVML: …") *and* a second line that matches none of them — "NVML library version: 595.91" — which both `install.sh`'s bash filter and `hardware_policy._nvidia_smi_gpu_names()` were reading as a real GPU product name. Chosing the GPU build plan on a driver that cannot currently build or run CUDA is what sent one field install into a doomed from-source CUDA build that failed partway through — which is what "the GUI wizard did not finish" actually was; the wizard has no build timeout of its own, `install.sh` really did exit non-zero. Fixed by checking nvidia-smi's *exit code* first in both places (reliable on both failure shapes) rather than only pattern-matching its stdout text.
- **`hardware_policy.py` also treated a bare `rocm-smi` binary as a working AMD GPU** — installed as part of a generic driver-utils package with no AMD hardware behind it, it still errors ("Driver not initialized (amdgpu not found in modules)") the moment it is actually queried. Fixed to require a real product-name answer, not just the binary existing.
- `install.sh`'s from-source CUDA build also used to fall back to `CMAKE_CUDA_ARCHITECTURES=native` when nvidia-smi's `compute_cap` query came back empty — but "native" makes `nvcc` re-probe the GPU through the same driver stack nvidia-smi just failed to reach, so it inherits the exact same failure. Falls back to the same known-good architecture list the CI-built GPU pack uses (`61;75;86;89`, Pascal–Ada) instead.
- **Known gap, not yet closed**: even after these fixes, a portable install that correctly lands on `--cpu-only` (driver unverifiable, or a genuinely weak/absent GPU) has no path back to GPU acceleration *during* that install — `install.sh` and `eli_setup.sh` have no awareness of the CI-built `gpu-packs` release at all; only the runtime "Install CUDA/Vulkan GPU pack" button in the Startup Model Selection dialog does. `install.sh` now at least tells the user that button exists instead of going quiet about GPU acceleration entirely. Teaching the portable installer to try the GPU pack itself (matching what the frozen/AppImage runtime already does automatically) is the real fix and is still open.

GPU pack (`packaging/pyinstaller/eli_gpu_pack.py`, `core/gpu_pack_runtime.py`)

Portable/AppImage builds download a CUDA or Vulkan build of `llama-cpp-python` per machine (the bundled runtime is CPU-only, safe everywhere) and shadow the bundled copy via `sys.path`.

- **`gpu_pack_looks_installed()`** is the trust-first check every normal boot uses: marker file + backend metadata + native libs present on disk, no subprocess, no GPU/driver contact. Only when this can't confirm the pack does anything fall through to `gpu_pack_operational()`, which live-probes offload in a throwaway subprocess — correct at install time, but a transient driver hiccup or busy GPU used to make that live probe fail even with a perfectly good pack installed, and any caller treating a failed live probe as "not installed" deleted a working pack and re-downloaded it on the very next launch. The cheap check is what stops that loop.
- **`activate_gpu_pack_runtime()`** preloads native libs, purges cached `llama_cpp`, calls `llama_backend_init()`, and verifies offload in-process — so a pack that verified in a subprocess actually works in the AppImage GUI process (Intel Iris Xe Vulkan is the case that needed this).
- `try_activate_gpu_pack()` (`gpu_pack_runtime.py`) delegates entirely to `activate_gpu_pack_runtime()`'s own (correctly cheap-first) gating rather than duplicating a check — a duplicate with reversed operand order here used to force the subprocess probe on every single GUI launch regardless of whether the marker file already said the pack was good.
- Writes `runtime/gpu/.gpu_pack_ok` + `.gpu_pack.json` (backend, version) on success; respects `runtime/.gpu_choice` for CPU opt-out.

Load-probe cache (`core/load_probe.py`)

`_gpu_identity()` memoizes a `name|total_mb` string per process so every cache lookup and record doesn't shell out to `nvidia-smi`. A detection **exception** (not a clean "no GPU found") no longer gets memoized as `"cpu"` — that used to permanently mislabel the process's load-probe cache identity for the rest of the session after one transient early-boot failure, even after the GPU became detectable. `"cpu"` is still

returned for that one call (a safe, conservative answer right now); the next call gets a real chance to detect the GPU.

The probe used to test the raw requested layer count regardless of MoE. For a mixture-of-experts model too big to fit as requested, the loader always offloads with experts in RAM and every layer on the GPU instead — a completely different, much lighter VRAM footprint — so the probe was proving a configuration that is never actually attempted, thrashing the full timeout budget (180s on an 8 GB card) every launch instead of the few seconds a real MoE load takes. It now checks `moe_offload.plan_for_load()` first and, when a plan applies, probes that configuration (the child subprocess wraps its `Llama(...)` call in `moe_offload.expert_offload_params()` the same way the real loaders do) — proven in 31s against a live model instead of an "unproven" 180s timeout.

That fixed WHAT is tested but not the timeout BUDGET, which still charged the full file size at the VRAM-upload rate (`_TIMEOUT_PER_GB_S`, calibrated for synchronous per-layer VRAM upload) — for a 24GB mixture-of-experts model that estimate came to ~294s, so the probe was still capped by the same 180s ceiling as everything else and looked "unproven" every launch regardless of the fix above. `_probe_budget_sizes_gb()` now separates the VRAM-bound share (the small resident portion under MoE, `moe_offload.plan_for_load().resident_gb`) from the rest of the file, which only pays a cheap disk-page-in rate (`_TIMEOUT_PER_GB_DISK_S`); a MoE model gets its own, higher ceiling (`_TIMEOUT_MOE_CEILING_S`, 240s) since the estimate is now trustworthy rather than a known-wrong number capped at the same limit as everything else. A genuinely dense model of the same size is unaffected — the two size shares collapse to the same value with no MoE plan, and the calculation reduces exactly to the original single-term formula.

`runtime_load_gap()` (`cognition/context_synthesiser.py`) only ever checked for `effective < requested` ("reduced," the classic VRAM-shortfall case) — a mixture-of-experts load *raises* `n_gpu_layers` above the request instead, and that case reported "loaded exactly as requested" for a load that did not match the request at all. It now reports a separate `raised` dict, and every surface that reads it (`GPU_STATUS`, `EXPLAIN_COGNITION_RUNTIME`'s `_inference_runtime_lines`, and `live_runtime_brief()` which grounds casual/phatic replies) states plainly which layers' attention/shared weights are on the GPU, how much VRAM that is, how many GB of experts sit in RAM, and that the CPU processes those during generation — not "the rest run on CPU" (backwards under MoE: nothing was left off the GPU) and not silence. `live_runtime_brief()` returning "" for a MoE load specifically is why a phatic reply once stated a stale, invented layer count: the grounding it would have drawn from was empty.

Settings (`core/runtime_settings.py`, ~1150 LOC)

- `DEFAULTS` (the full settings schema) + `ENV_TO_KEY` (env-var overrides).
- `load_settings` / `save_settings` / `update_settings`.
- **Redistribution-aware**: `_migrate_legacy_keys` (schema evolution), `_resolve_relative_model_paths` + `_heal_model_paths` (fix stale absolute paths when the project moves machines), and `_portable_settings_for_sto` (strip machine-specific values before storage) — though personal values like `user_name` can still end up tracked.
- **`settings_file_was_corrupt_on_last_load()`** — a settings file that exists but fails to parse (truncated write, disk-full, a concurrent-write race) silently fell back to `DEFAULTS` with no signal that the fallback happened, indistinguishable from a normal first run with no file yet. Anything rendering settings to the user should check this rather than presenting defaults as if they were read from disk.

Paths (`core/paths.py`, 808 LOC)

Dev-vs-packaged path resolution: `is_frozen` / `_is_dev_mode`, `_find_project_root`, then `data_dir/config_dir/cache_dir/m` `db_dir/artifacts_dir/user_db_path/agent_db_path/memory_db_path`. A source checkout uses project-local `artifacts/` + `config/`. The packaged app and an installed one both use v2's own root: `~/.local/share/ELI_v2` (`artifacts/`, `config/`, `cache/`), `%LOCALAPPDATA%\ELI_v2` on Windows, `~/Library/Application Support/ELI_v2` on macOS. One import surface (`get_paths`) so nothing hardcodes locations.

Nothing of v2's goes in a generic place: `~/.local/share/eli`, `~/.config/eli`, `~/.cache/eli`, `~/.eli/` and `~/.eli_*` belong to no single ELI version, and v3, the first version (`eli-ai`), ELI Legal and ELI VR keep their own folders. Until 2.5.15 an installed v2 used the generic `eli` folders and the packaged app kept its cache in `~/.cache/eli`; some files had a generic or relative fallback (`artifacts/...` lands in whatever folder ELI was started from). Every writer now goes through `get_paths`. A file v2 once kept in a generic place (the labs notebook and workspaces in `~/.eli/`, `~/.eli_user_id`) is copied into its own folders once by `adopt_legacy_file` (copied, not moved: another version may still use the old place). `tests/test_frozen_install_redistributability.py` scans the code for generic and relative paths.

Launchers are v2's too: `~/.local/bin/eli-v2-run`, `eli-v2*.desktop` menu entries and the `eli-v2` icon. Setup and the uninstaller replace or remove only v2's own entries (named `eli-v2…`, or an older entry whose contents name v2); another version's launchers are never touched.

STT VRAM awareness

`local_whisper_stt` is VRAM-aware (GPU only on ≥12 GB cards, else CPU) so faster-whisper preloading on CUDA doesn't starve the main GGUF model's layer budget. `ELI_WHISPER_DEVICE` overrides; `ELI_WHISPER_GPU_MIN_MB` tunes the threshold. See `perception.md`.

Piper and LoRA device selection

Piper is a subprocess binary (`tts_piper/piper`), not onnxruntime-in-Python, and the shipped build is CPU-only for every vendor — no NVIDIA advantage to close there.

`eli/learning/lora_trainer._accelerator()` is the model for cross-vendor device selection generally: torch routes ROCm through the `torch.cuda` API (a HIP build answers `torch.cuda.is_available()`), so one code path serves NVIDIA and AMD, with the real vendor read from `torch.version.hip` and reported honestly rather than as "CUDA on a Radeon." Apple (mps) and Intel (xpu) are detected and honestly marked as unable to run that trainer. See `learning.md`.

Honest assessment

- **Strong:** genuinely adaptive and agnostic — free-VRAM-aware sizing, the compute-buffer reservation that prevents first-decode crashes, graceful GPU-layer fallback, filename→ctx adaptation, env/settings/relative-path healing for moving between machines, a single broker + lock so concurrency is correct, and (as of this release) a GPU pack that trusts a verified install instead of re-proving itself live on every boot. This is mature, hard-won infrastructure.
- **Weak / watch:**
  1. **Filename-based family detection** (chat template + ctx) is fragile: a model with an unconventional filename gets a default template + 32768 ctx, which can be wrong. A metadata/GGUF-header probe would be more robust than string matching.
  2. **Settings sprawl** — `DEFAULTS` is large with several overlapping keys (`n_gpu_layers`/`gpu_layers`, `n_ctx`/`context_size`) and migration logic.
  3. VRAM heuristics are empirically tuned around an 8GB card. The flat-guess fallbacks that used to apply uniformly regardless of actual card/RAM size are narrower now (known-model lookup for NVIDIA/Arc, uncapped budget for unified memory — see above), but an *unrecognized* NVIDIA/Arc model still lands on a conservative flat guess, and the live nvidia-smi/rocm-smi path (when it works) is still the only source of a true measured number rather than a factory-spec lookup or RAM-fraction estimate.
  4. `_portable_settings_for_storage` exists but isn't fully preventing personal values (e.g. `user_name`) from being persisted/committed — worth tightening for redistribution.
  5. The ~15 sites that bypass `hardware_profile` for direct `nvidia-smi` calls (see above) are still there — known, not yet routed through the shared profile.